

PROJECT REPORT

SMART FILE ORGANIZER

SUBMITTED BY-

Divya Sinha [26BCE11195]

PROJECT TITLE-

Smart File Organizer

LANGUAGE USED-

Python 3.x

SUBMITTED TO-

Dr. Anjali Mathur

SUBJECT-

Python essentials

INTRODUCTION

In today's digital time, users create and store a large number of files such as photos, documents, videos, songs, and software. However, after some time when these files pile up, it becomes difficult to organize them and find at the time of need.

The Smart File Organiser is a Python-based project developed to classify files according to their extensions and store them into suitable categories. This project helps users to keep their files organised and easy to access.

The system uses basic Python concepts like variables, loops, conditional statements and string operations.

PROBLEM STATEMENT

Users store different types of files in a single location. Finding a particular file among lots of mixed files can be time-consuming and tedious.

The objective of this project is to identify file types based on file extensions and classify them into categories such as Images, Documents, Audio, Videos, Programming Files and Others.

FUNCTIONAL REQUIREMENTS

The Smart File Organiser performs the following functions:

1. Accepts file names from the user.
2. Identifies the file type using file extensions.

3. Classify image files.
4. Classify document files.
5. Classify audio files.
6. Classify video files.
7. Classify programming files.
8. Displays the category of each file.
9. Maintains a count of files in each category.
10. Displays the final organisation report.

DESIRABLE REQUIREMENTS

Performance

The system should process files quickly.

Reliability

The program should correctly identify file categories.

Usability

Easy for users.

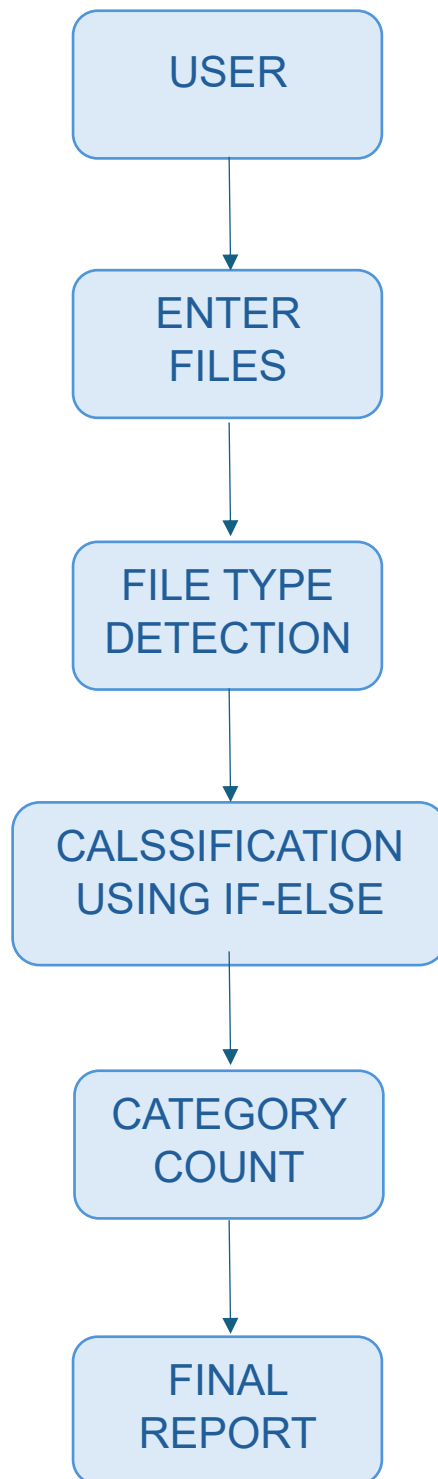
Maintainability

Additional file types can be added easily later.

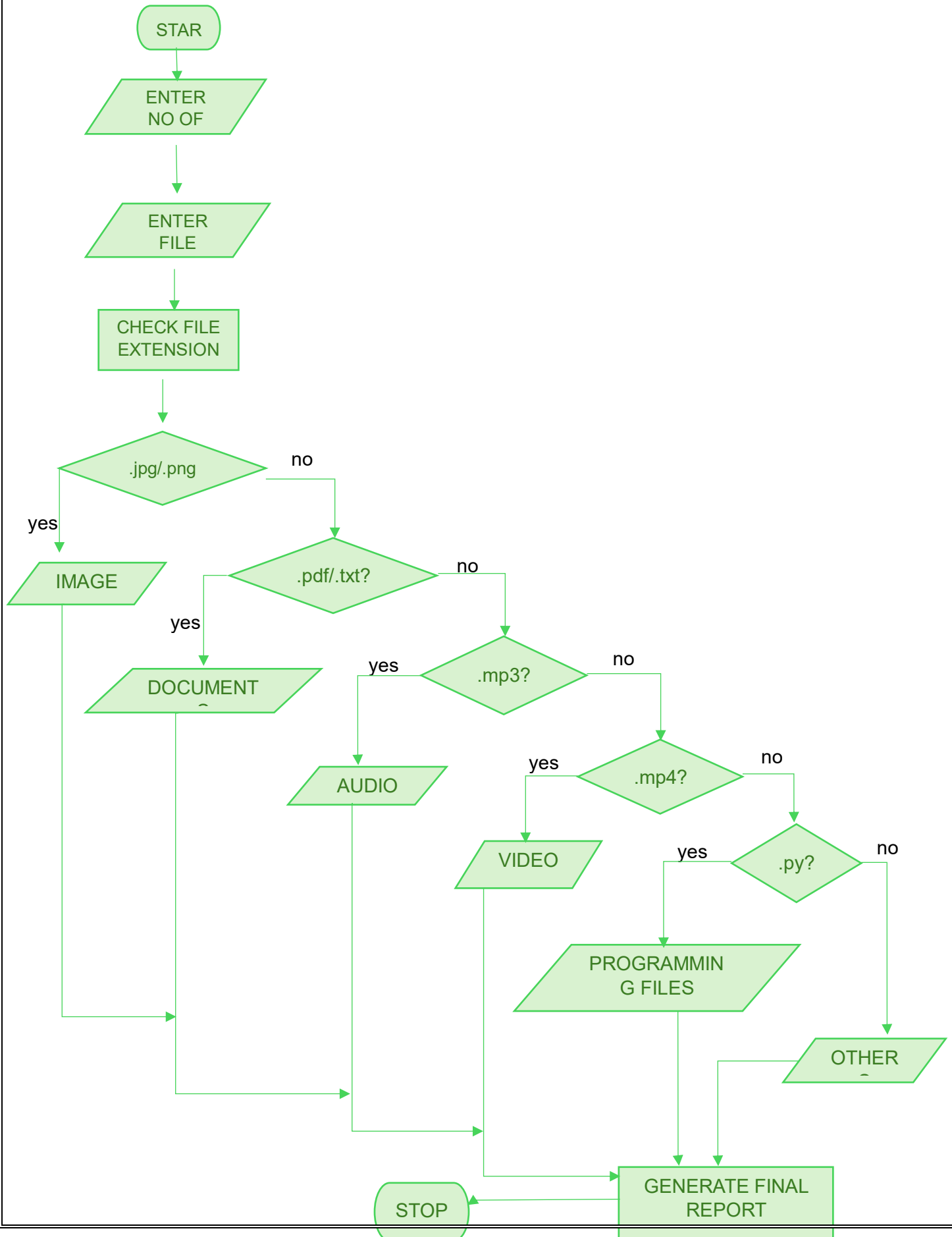
Portability

Can run on any system supporting Python.

SYSTEM ARCHITECTURE



WORKFLOW DIAGRAM



Implementation Details

Software Requirements

- Python 3.x
- Windows Operating System
- Text Editor (VS Code, IDLE, or PyCharm)

Concepts Used

- Variables
- Input and Output
- If-Elif-Else
- For Loop
- String Operations
- Counters

Algorithm

1. Start the program.
2. Accept the number of files.
3. Read file names one by one.
4. Check the extension of each file.
5. Assign a category.
6. Increase category count.
7. Display category information.
8. Generate the final report.
9. Stop the program.

OUTPUT SCREENSHOT

```
===== RESTART: D:\Programming
=====
SMART FILE ORGANISER
=====
Enter number of files: 3

File 1
Enter file name: p1.txt
p1.txt -> Documents Folder

File 2
Enter file name: interview.wav
interview.wav -> Audio Folder

File 3
Enter file name: cat.zip
cat.zip -> Others Folder

=====
ORGANISATION REPORT
=====
Total Files Entered : 3
Image Files          : 0
Document Files       : 1
Audio Files          : 1
Video Files          : 0
Other Files          : 1

Thank You For Using Smart File Organiser
>>> |
```

```
===== RESTART: D:\Programming
=====
SMART FILE ORGANISER
=====
Enter number of files: -1

=====
ORGANISATION REPORT
=====
Total Files Entered : 0
Image Files          : 0
Document Files       : 0
Audio Files          : 0
Video Files          : 0
Other Files          : 0

Thank You For Using Smart File Organiser
>>> |
```

```
===== RESTART: D:\Programming
=====
SMART FILE ORGANISER
=====
Enter number of files: 5

File 1
Enter file name: photo.jpg
photo.jpg -> Images Folder

File 2
Enter file name: report.pdf
report.pdf -> Documents Folder

File 3
Enter file name: song.mp3
song.mp3 -> Audio Folder

File 4
Enter file name: video.mp4
video.mp4 -> Videos Folder

File 5
Enter file name: program.py
program.py -> Programming Folder

=====
ORGANISATION REPORT
=====
Total Files Entered : 5
Image Files          : 1
Document Files       : 1
Audio Files          : 1
Video Files          : 1
Other Files          : 0

Thank You For Using Smart File Organiser
>>> |
```

TESTING APPROACH

Test Case	Input	Expected Output	Result
1.	Image.jpg	Images folder	Pass
2.	File.pdf	Documents folder	Pass
3.	Song.mp3	Audio folder	Pass
4.	Movie.mp4	Videos folder	Pass
5.	Abc.py	Programming folder	Pass
6.	Xyz.xyz	Others folder	pass

CHALLENGES FACED

1. Identifying correct file extensions.
2. Handling multiple file categories.
3. Maintaining category counts.
4. Designing a simple and user-friendly interface.
5. Ensuring accurate classification using conditional statements.

CONCLUSION

The Smart File Organiser successfully classifies files based on their extensions using Python. This project demonstrates the use of conditional statements, loops, variables and basic programming concepts. It helps improve file management and can be further developed for advanced file organisation systems.